ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА



Лабораторная работа №3

по дисциплине «Инженерия данных»

Выполнил:

Студент группы J3311
Поладов Селим Бегенджович
ИСУ: 467128

Санкт-Петербург
2026

1. Развертывание RustFS

Цель работы в том, чтобы посчитать навыки, пары и тройки навыков своего варианта двумя независимыми способами по данным из объектного хранилища, сверить результаты, сравнить подходы по времени и памяти и найти навыки, которые связаны с зарплатой и различаются по регионам. Вариант 9, программист, разработчик на мобильных платформах.

Работа выполнена 21 октября 2026 года на macOS, процессор с 12 ядрами, 24 ГБ памяти, Docker Desktop. RustFS 1.0.0 поднят по профилю home из стенда второго домашнего задания. Один процесс работает с четырьмя папками-дисками vol1, vol2, vol3 и vol4, схема кодирования EC 2+2. Папки лежат в локальном каталоге data, ключи доступа в файле .env, который не попадает в отчет и репозиторий. S3 API доступен на порту 9010, консоль на порту 9011.

```
services:
  rustfs-home:
    image: rustfs/rustfs:1.0.0
    container_name: hh-lake-rustfs
    profiles: ["home"]
    env_file: .env
    environment:
      RUSTFS_VOLUMES: /data/vol{1...4}
      RUSTFS_UNSAFE_BYPASS_DISK_CHECK: "true"
      RUSTFS_STORAGE_CLASS_STANDARD: "EC:2"
    ports: ["127.0.0.1:9010:9000", "127.0.0.1:9011:9001"]
    volumes: ["./data:/data"]
```

```
$ docker compose --profile home up -d rustfs-home
 Network hh-lake_default  Creating
 Network hh-lake_default  Created
 Container hh-lake-rustfs  Creating
 Container hh-lake-rustfs  Created
 Container hh-lake-rustfs  Starting
 Container hh-lake-rustfs  Started
$ docker compose --profile home ps --format "{{.Name}} {{.Image}} {{.Status}} {{.Ports}}"
hh-lake-rustfs rustfs/rustfs:1.0.0 Up 5 seconds 127.0.0.1:9010->9000/tcp, 127.0.0.1:9011-
>9001/tcp
$ docker exec hh-lake-rustfs rustfs --version | head -1
rustfs 1.0.0
$ curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:9010/health
200
```

Рис. 1. Запуск контейнера и проверка RustFS

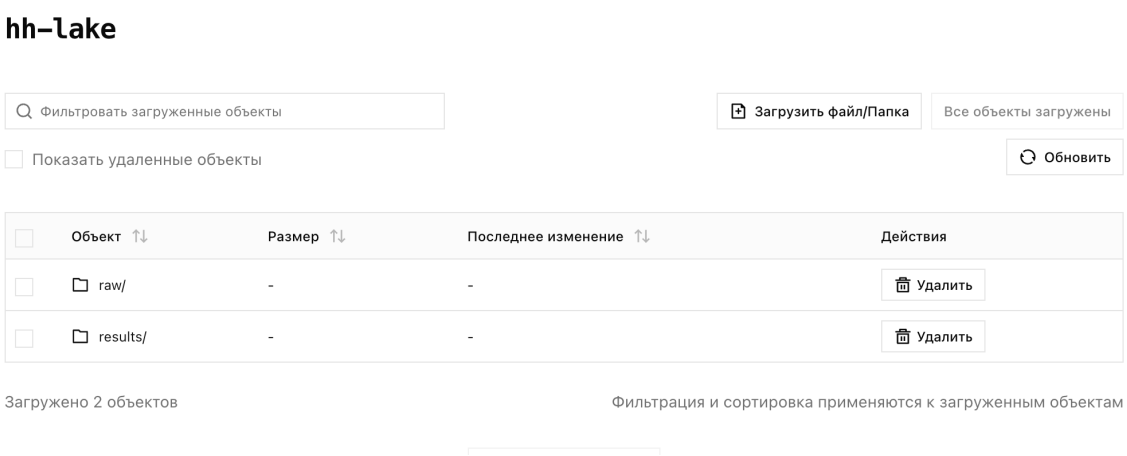


Рис. 2. Бакет hh-lake в консоли RustFS

2. Загрузка данных

Используется предоставленный файл it_vacancies_2024.csv с IT-вакансиями hh.ru за 2024 год. В нем 229388 вакансий, 23 столбца и 847619384 байта. Каждая запись занимает ровно одну строку файла, всего 229389 строк вместе с заголовком. Поэтому файл можно делить на части по байтам и выравнивать части по переводу строки.

Скрипт upload.py загрузил файл в бакет hh-lake как объект raw/it_vacancies_2024.csv через boto3. После загрузки объект прочитан обратно, SHA256 совпала с локальным файлом.

В бакет загружен полный датасет. Вакансии варианта отбираются при обработке одним и тем же условием, которое отдельно написано на Python для MapReduce и на SQL для DuckDB. Так условие варианта проверяется в двух независимых реализациях.

```
$ .venv/bin/python scripts/upload.py
created bucket hh-lake
uploaded s3://hh-lake/raw/it_vacancies_2024.csv bytes=847619384 time=5.1s
head content_length=847619384
sha256_local=303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
sha256_s3=303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
match=True
```

Рис. 3. Загрузка датасета и проверка контрольной суммы

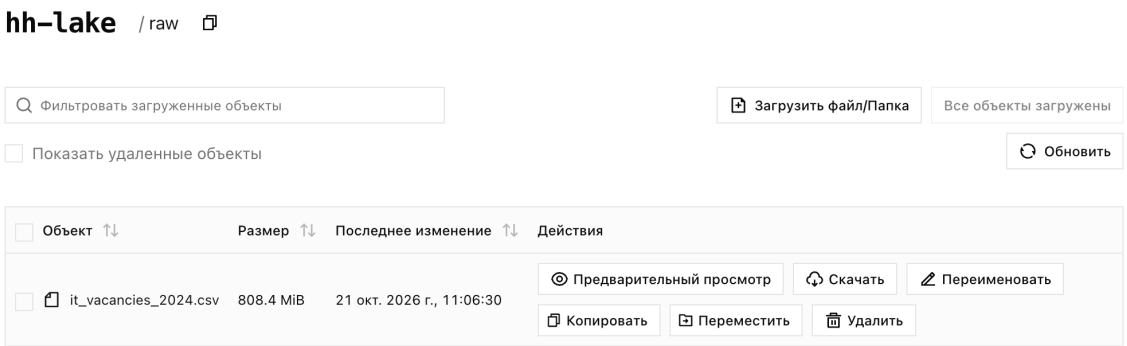


Рис. 4. Объект с датасетом в бакете hh-lake

3. Подсчет навыков, пар и троек

Условие варианта и навыки

Вакансия относится к варианту, если ее роль hh равна `Программист, разработчик`, а в названии есть мобильная платформа. Название приводится к нижнему регистру. С начала слова ищутся android, андроид, мобильн, flutter, флаттер, xamarin, swift, react native, objective-c и kotlin multiplatform. Отдельными словами ищутся ios, айос, mobile, ionic и kmp. Граница слова задана явно классом символов, поэтому одно и то же регулярное выражение одинаково работает в Python и в DuckDB. Kotlin без мобильной платформы в условие не входит, потому что на Kotlin пишут и серверные приложения, например в вакансиях Java/Kotlin Developer. Условию соответствует 3771 вакансия, у 2794 из них заполнены ключевые навыки.

```
role = 'Программист, разработчик' AND regexp_matches(lower(name),
'(^|[^a-za-яё0-9])(android|андроид|мобильн|flutter|флаттер|xamarin|swift|react.native|ob⌋
jective.c|kotlin
multiplatform)|(^|[^a-za-яё0-9])(ios|айос|mobile|ionic|kmp)([^a-za-яё0-9]|$)')
```

Навыки берутся из столбца key_skills. Строка делится по точке с запятой, пробелы по краям убираются, навык приводится к нижнему регистру, повторы внутри вакансии удаляются, навыки сортируются. Нижний регистр нужен, потому что один навык записан по-разному, например Kotlin, kotlin и KOTLIN. Пары и тройки это сочетания разных навыков одной вакансии в порядке сортировки. Частота сочетания равна числу вакансий, в которых оно есть.

Собственный MapReduce

Программа mapreduce.py повторяет схему map, shuffle и reduce. Объект делится по байтам на 26 сплитов по 32 МиБ. Каждая map-задача скачивает свой сплит из RustFS запросом GET с заголовком Range и читает ответ кусками по 1 МиБ. Строка принадлежит тому сплиту, в котором она начинается, как в LineRecordReader в Hadoop. Задача пропускает хвост строки предыдущего сплита и дочитывает свою последнюю строку за границей сплита. Скрипт test_splits.py проверил это на локальной копии файла для сплитов 1000003, 7340043 и 33554432 байта. Каждая из 229389 строк обработана ровно один раз.

Функция map разбирает строку CSV, проверяет условие варианта и для каждой вакансии выдает ключи всех сочетаний из одного, двух и трех навыков со значением 1. Комбайнер суммирует значения одинаковых ключей внутри map-задачи. Это допустимо, потому что сумма ассоциативна и коммутативна. Ключ попадает в одну из четырех партиций по crc32, партиция сортируется и пишется на локальный диск. Reduce-задача сливает отсортированные партиции всех map-задач и суммирует значения одного ключа. Задачи работают отдельными процессами, одновременно не больше восьми. Выход задачи фиксируется переименованием каталога только после успешной попытки, упавшая попытка перезапускается до четырех раз.

Предфильтр пропускает разбор CSV для строк, в байтах которых нет текста роли варианта. Такая строка точно не проходит условие, поэтому результат не меняется. Текст роли есть в 87803 строках из 229389, остальные строки не разбираются.

```
for pos, line in lines:                          # строки своего сплита
    if line is None or ROLE_BYTES not in line:   # предфильтр
        continue
    row = next(csv.reader([line.decode('utf-8')]))
    if not in_variant(row):
        continue
    sk = skills_of(row)                          # нормализованные навыки
    for k in (1, 2, 3):
        for combo in combinations(sk, k):
```

```
            key = (k,) + combo
            parts[part_of(key, reducers)][key] += 1   # комбайнер
```

```
$ .venv/bin/python scripts/mapreduce.py --upload
0.06s input s3://hh-lake/raw/it_vacancies_2024.csv bytes=847619384 chunk_mib=32 splits=26
workers=8 reducers=4 combiner=True prefilter=True stream=True
4.73s map phase done bytes_read=854173009 map_input_records=229389 map_output_records=191397
spilled_records=138585 variant_records=3771 variant_records_with_skills=2794
5.01s reduce phase done reduce_input_records=138585 reduce_output_records=74904
5.01s k=1 distinct=1033 total=15255
5.01s k=1 top1 android vacancies=1180
5.01s k=1 top2 kotlin vacancies=1073
5.01s k=1 top3 ios vacancies=905
5.04s uploaded s3://hh-lake/results/mapreduce/skills_k1.csv bytes=20917
5.05s k=2 distinct=12308 total=48175
5.05s k=2 top1 android + kotlin vacancies=784
5.05s k=2 top2 ios + swift vacancies=561
5.05s k=2 top3 android sdk + kotlin vacancies=514
5.12s uploaded s3://hh-lake/results/mapreduce/skills_k2.csv bytes=333496
5.18s k=3 distinct=61563 total=127967
5.18s k=3 top1 android + android sdk + kotlin vacancies=392
5.18s k=3 top2 android + java + kotlin vacancies=364
5.18s k=3 top3 android sdk + java + kotlin vacancies=289
5.23s uploaded s3://hh-lake/results/mapreduce/skills_k3.csv bytes=2199908
5.23s job finished
```

Рис. 5. Запуск MapReduce с записью результатов в hh-lake

Счетчики задачи согласованы между собой. На вход map пришли 229389 строк вместе с заголовком, условию соответствуют 3771 вакансия. Map выдал 191397 пар ключ и значение, это сумма 15255, 48175 и 127967 по трем размерам сочетаний. Комбайнер сократил данные шаффла до 138585 записей. Reduce выдал 74904 ключа, это 1033 навыка, 12308 пар и 61563 тройки.

SQL в DuckDB

Скрипт duckdb_skills.py читает тот же объект s3://hh-lake/raw/it_vacancies_2024.csv через расширение httpfs. Условие варианта и нормализация навыков записаны в SQL. Пары и тройки получаются соединением таблицы навыков с самой собой по вакансии с условием `a.skill < b.skill`.

```
CREATE TABLE seg AS
SELECT vacancy_id, area, name, experience, schedule,
       salary_from, salary_to, currency, gross,
       list_sort(list_distinct(list_filter(
           list_transform(string_split(key_skills, ';'), x -> lower(trim(x))),
           x -> x <> ''))) AS skills
FROM read_csv('s3://hh-lake/raw/it_vacancies_2024.csv',
              header = true, all_varchar = true)
WHERE role = 'Программист, разработчик'
  AND regexp_matches(lower(name), '<шаблон варианта>');
CREATE TABLE ex AS SELECT vacancy_id, unnest(skills) AS skill FROM seg;
SELECT a.skill AS skill_1, b.skill AS skill_2, c.skill AS skill_3,
```

```
       count(*) AS vacancies
FROM ex a JOIN ex b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
          JOIN ex c ON b.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY ALL ORDER BY vacancies DESC, skill_1, skill_2, skill_3;
```

```
$ .venv/bin/python scripts/duckdb_skills.py --upload
4.50s variant_vacancies=3771 with_skills=2794
4.51s k=1 distinct=1033 total=15255
4.51s k=1 top1 android vacancies=1180
4.51s k=1 top2 kotlin vacancies=1073
4.51s k=1 top3 ios vacancies=905
4.53s uploaded s3://hh-lake/results/duckdb/skills_k1.csv
4.54s k=2 distinct=12308 total=48175
4.54s k=2 top1 android + kotlin vacancies=784
4.54s k=2 top2 ios + swift vacancies=561
4.54s k=2 top3 android sdk + kotlin vacancies=514
4.57s uploaded s3://hh-lake/results/duckdb/skills_k2.csv
4.60s k=3 distinct=61563 total=127967
4.60s k=3 top1 android + android sdk + kotlin vacancies=392
4.60s k=3 top2 android + java + kotlin vacancies=364
4.60s k=3 top3 android sdk + java + kotlin vacancies=289
4.66s uploaded s3://hh-lake/results/duckdb/skills_k3.csv
4.66s done
```

Рис. 6. Тот же расчет в DuckDB

Результаты и рост числа сочетаний

Самые частые навыки варианта android в 1180 вакансиях, kotlin в 1073, ios в 905, git в 834 и swift в 717. Самые частые пары android + kotlin в 784 вакансиях, ios + swift в 561 и android sdk + kotlin в 514. Самые частые тройки android + android sdk + kotlin в 392 вакансиях, android + java + kotlin в 364 и android sdk + java + kotlin в 289. Рынок делится на два нативных стека, Android на Kotlin и Java и iOS на Swift, и меньший сегмент кроссплатформенной разработки на Flutter и React Native.

Различных сочетаний становится намного больше с ростом размера. Навыков 1033, пар 12308, троек 61563. Сумма частот растет так же, 15255, 48175 и 127967. Причина в числе сочетаний внутри одной вакансии. Вакансия с n навыками дает $n(n-1)/2$ пар и $n(n-1)(n-2)/6$ троек. При медиане 5 навыков это 10 пар и 10 троек, а вакансия с 28 навыками дает 378 пар и 3276 троек. Поэтому 9,6 процента вакансий, где навыков не меньше 10, дают 68,3 процента всех троек. Суммы $C(n, k)$ по вакансиям точно совпадают с итогами MapReduce. Большинство длинных сочетаний редкие, один раз встречаются 46 процентов навыков, 55 процентов пар и 61 процент троек, поэтому число различных ключей растет быстрее, чем сумма частот.

```
$ .venv/bin/python scripts/combinatorics.py
skills_per_vacancy vacancies=2794 min=1 median=5.0 mean=5.46 max=28
k=1 combinations=15255
k=2 combinations=48175
k=3 combinations=127967
n=1 vacancies=101 pairs_each=0 triples_each=0 triples=0 triples_share=0.0
n=2 vacancies=280 pairs_each=1 triples_each=0 triples=0 triples_share=0.0
n=3 vacancies=381 pairs_each=3 triples_each=1 triples=381 triples_share=0.3
n=4 vacancies=472 pairs_each=6 triples_each=4 triples=1888 triples_share=1.5
n=5 vacancies=493 pairs_each=10 triples_each=10 triples=4930 triples_share=3.9
n=6 vacancies=288 pairs_each=15 triples_each=20 triples=5760 triples_share=4.5
n=7 vacancies=210 pairs_each=21 triples_each=35 triples=7350 triples_share=5.7
n=8 vacancies=183 pairs_each=28 triples_each=56 triples=10248 triples_share=8.0
n=9 vacancies=119 pairs_each=36 triples_each=84 triples=9996 triples_share=7.8
n=10 vacancies=75 pairs_each=45 triples_each=120 triples=9000 triples_share=7.0
n=11 vacancies=55 pairs_each=55 triples_each=165 triples=9075 triples_share=7.1
n=12 vacancies=42 pairs_each=66 triples_each=220 triples=9240 triples_share=7.2
n=13 vacancies=17 pairs_each=78 triples_each=286 triples=4862 triples_share=3.8
n=14 vacancies=26 pairs_each=91 triples_each=364 triples=9464 triples_share=7.4
n=15 vacancies=14 pairs_each=105 triples_each=455 triples=6370 triples_share=5.0
n=16 vacancies=10 pairs_each=120 triples_each=560 triples=5600 triples_share=4.4
n=17 vacancies=11 pairs_each=136 triples_each=680 triples=7480 triples_share=5.8
n=18 vacancies=6 pairs_each=153 triples_each=816 triples=4896 triples_share=3.8
n=20 vacancies=2 pairs_each=190 triples_each=1140 triples=2280 triples_share=1.8
n=21 vacancies=3 pairs_each=210 triples_each=1330 triples=3990 triples_share=3.1
n=22 vacancies=2 pairs_each=231 triples_each=1540 triples=3080 triples_share=2.4
n=26 vacancies=1 pairs_each=325 triples_each=2600 triples=2600 triples_share=2.0
n=27 vacancies=1 pairs_each=351 triples_each=2925 triples=2925 triples_share=2.3
n=28 vacancies=2 pairs_each=378 triples_each=3276 triples=6552 triples_share=5.1
skills_10_or_more vacancies_share=9.6 triples_share=68.3
k=1 distinct=1033 seen_once=479 seen_once_share=46.4
k=2 distinct=12308 seen_once=6747 seen_once_share=54.8
k=3 distinct=61563 seen_once=37524 seen_once_share=61.0
```

Рис. 7. Число сочетаний в зависимости от числа навыков вакансии

Результаты для всех трех размеров сочетаний обоими способами лежат в бакете hh-lake в папках results/mapreduce и results/duckdb.

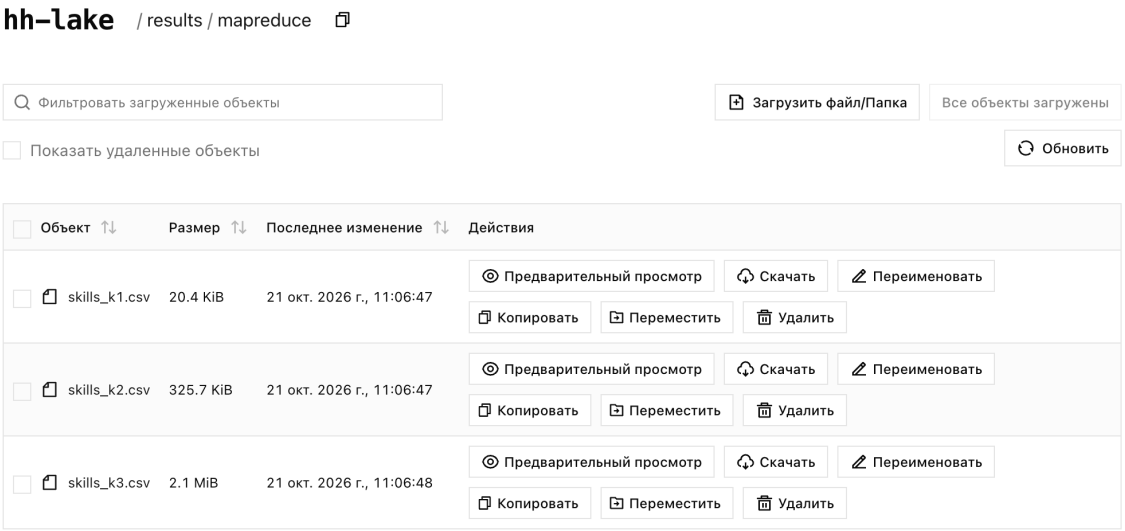


Рис. 8. Результаты MapReduce в бакете hh-lake

```
$ .venv/bin/python scripts/s3ls.py
2026-10-21 11:06:30 847619384 s3://hh-lake/raw/it_vacancies_2024.csv
2026-10-21 11:14:45 11155 s3://hh-lake/results/analysis/regions_top.csv
2026-10-21 11:14:40 15130 s3://hh-lake/results/analysis/salary_uplift_k2.csv
2026-10-21 11:14:40 16838 s3://hh-lake/results/analysis/salary_uplift_k3.csv
2026-10-21 11:06:52 19891 s3://hh-lake/results/duckdb/skills_k1.csv
2026-10-21 11:06:52 321529 s3://hh-lake/results/duckdb/skills_k2.csv
2026-10-21 11:06:52 2141120 s3://hh-lake/results/duckdb/skills_k3.csv
2026-10-21 11:06:47 20917 s3://hh-lake/results/mapreduce/skills_k1.csv
2026-10-21 11:06:47 333496 s3://hh-lake/results/mapreduce/skills_k2.csv
2026-10-21 11:06:48 2199908 s3://hh-lake/results/mapreduce/skills_k3.csv
total_bytes=852699368
```

Рис. 9. Все объекты бакета hh-lake после выполнения работы

4. Верификация результатов

Скрипт verify.py скачивает из hh-lake результаты обоих способов и сравнивает все строки, то есть сочетания и их частоты. Совпали все 1033 строки навыков, 12308 строк пар и 61563 строки троек, суммы и контрольные хеши. Сами файлы отличаются по байтам только форматом CSV. У Python окончания строк CRLF, а DuckDB берет в кавычки значения с решеткой, например c#. Поэтому сравниваются разобранные значения, а не байты файлов.

```
$ .venv/bin/python scripts/verify.py
k=1 mapreduce rows=1033 sum=15255 sha=f4e6f2d80cee9847
k=1 duckdb rows=1033 sum=15255 sha=f4e6f2d80cee9847
k=1 only_mapreduce=0 only_duckdb=0 count_diff=0 equal=True
k=2 mapreduce rows=12308 sum=48175 sha=1796d4493b32b06a
k=2 duckdb rows=12308 sum=48175 sha=1796d4493b32b06a
k=2 only_mapreduce=0 only_duckdb=0 count_diff=0 equal=True
k=3 mapreduce rows=61563 sum=127967 sha=9643c72c687f21b9
k=3 duckdb rows=61563 sum=127967 sha=9643c72c687f21b9
k=3 only_mapreduce=0 only_duckdb=0 count_diff=0 equal=True
all_equal=True
```

Рис. 10. Полное совпадение результатов MapReduce и DuckDB

Отдельно проверен отказ маппера. При запуске с ключом `--fail-map 5` процесс map-задачи 5 на первой попытке завершается посреди сплита с кодом 17 и оставляет недописанный выход в каталоге попытки. Планировщик запускает задачу повторно, вторая попытка проходит. Недописанный выход не используется, потому что зафиксирован только каталог успешной попытки. Результаты побайтно совпали с обычным запуском.

```
$ .venv/bin/python scripts/mapreduce.py --fail-map 5 --job failtest --out mapreduce_failtest
0.05s input s3://hh-lake/raw/it_vacancies_2024.csv bytes=847619384 chunk_mib=32 splits=26
workers=8 reducers=4 combiner=True prefilter=True stream=True
1.08s map task 5 attempt 1 failed exit=17
1.08s map task 5 restarted attempt 2
2.10s map task 5 attempt 2 succeeded
4.48s map phase done bytes_read=854173009 map_input_records=229389 map_output_records=191397
spilled_records=138585 variant_records=3771 variant_records_with_skills=2794
4.74s reduce phase done reduce_input_records=138585 reduce_output_records=74904
4.75s k=1 distinct=1033 total=15255
4.75s k=1 top1 android vacancies=1180
4.75s k=1 top2 kotlin vacancies=1073
4.75s k=1 top3 ios vacancies=905
4.75s k=2 distinct=12308 total=48175
4.75s k=2 top1 android + kotlin vacancies=784
4.75s k=2 top2 ios + swift vacancies=561
4.75s k=2 top3 android sdk + kotlin vacancies=514
4.81s k=3 distinct=61563 total=127967
4.81s k=3 top1 android + android sdk + kotlin vacancies=392
4.81s k=3 top2 android + java + kotlin vacancies=364
4.81s k=3 top3 android sdk + java + kotlin vacancies=289
4.81s job finished
$ ls tmp/mr/failtest | grep map-005
map-005
map-005.attempt-1
$ cat tmp/mr/failtest/map-005.attempt-1/part-000.jsonl
incomplete
$ shasum -a 256 results/mapreduce/*.csv results/mapreduce_failtest/*.csv
79c19a302b223337d86a0cc4d5de9b56c32893014d9dfc646f8b55d9d18ccb7a
results/mapreduce/skills_k1.csv
2fbb0cb56c472f162f2c184e7045ead2ab16727b84a9cab53a8ca359b0840f71
results/mapreduce/skills_k2.csv
a24bbfe405192ab0d24db05d8d02b5924747c376c716b14be0ef473ed533b30b
results/mapreduce/skills_k3.csv
79c19a302b223337d86a0cc4d5de9b56c32893014d9dfc646f8b55d9d18ccb7a
results/mapreduce_failtest/skills_k1.csv
2fbb0cb56c472f162f2c184e7045ead2ab16727b84a9cab53a8ca359b0840f71
results/mapreduce_failtest/skills_k2.csv
a24bbfe405192ab0d24db05d8d02b5924747c376c716b14be0ef473ed533b30b
results/mapreduce_failtest/skills_k3.csv
```

Рис. 11. Отказ map-задачи и ее повторный запуск

5. Профилирование и оптимизация

Скрипт profile.py запускает команду пять раз. Время измеряется от запуска до завершения процесса. Память измеряется каждые 20 мс как сумма RSS процесса и всех его потомков, берется максимум за запуск. Для MapReduce это важно, потому что map- и reduce-задачи работают отдельными процессами. Все запуски читают данные из RustFS.

Конфигурация	Время, с, медиана и разброс	Пик памяти, МиБ, медиана и разброс
MapReduce до оптимизации, 1 процесс, сплит целиком в памяти, без комбайнера и предфильтра	14,58 (14,46 и 15,47)	278,0 (259,3 и 280,2)
MapReduce, 1 процесс, потоковое чтение, комбайнер, предфильтр	11,74 (11,36 и 12,31)	163,2 (160,9 и 164,1)
MapReduce, 8 процессов, без остальных оптимизаций	5,85 (5,77 и 6,53)	1388,2 (1324,1 и 1575,0)
MapReduce после оптимизации, 8 процессов, потоковое чтение, комбайнер, предфильтр	5,70 (5,28 и 6,06)	718,4 (669,8 и 735,3)
DuckDB, SQL по тому же объекту	4,61 (4,49 и 4,82)	440,3 (417,0 и 477,1)
Только чтение объекта из RustFS в 8 потоков без обработки	3,78 (3,67 и 3,81)	113,6 (111,9 и 117,9)

Таблица 1. Время и пик памяти, по пять запусков на конфигурацию, в скобках наименьшее и наибольшее значение

```
$ .venv/bin/python scripts/profile.py mr_base 5 -- .venv/bin/python scripts/mapreduce.py --workers 1 --no-
combiner --no-prefilter --no-stream --job p_base --out p_base
mr_base run 1 time=14.58s peak_rss=279.3MiB
mr_base run 2 time=15.38s peak_rss=277.7MiB
mr_base run 3 time=15.47s peak_rss=259.3MiB
mr_base run 4 time=14.46s peak_rss=278.0MiB
mr_base run 5 time=14.56s peak_rss=280.2MiB
mr_base median time=14.58s peak_rss=278.0MiB
$ .venv/bin/python scripts/profile.py mr_seq_opt 5 -- .venv/bin/python scripts/mapreduce.py --workers 1 --
job p_seq --out p_seq
mr_seq_opt run 1 time=11.36s peak_rss=162.3MiB
mr_seq_opt run 2 time=11.74s peak_rss=164.1MiB
mr_seq_opt run 3 time=12.31s peak_rss=163.4MiB
mr_seq_opt run 4 time=11.72s peak_rss=163.2MiB
mr_seq_opt run 5 time=12.20s peak_rss=160.9MiB
mr_seq_opt median time=11.74s peak_rss=163.2MiB
$ .venv/bin/python scripts/profile.py mr_par 5 -- .venv/bin/python scripts/mapreduce.py --workers 8 --no-
combiner --no-prefilter --no-stream --job p_par --out p_par
mr_par run 1 time=6.53s peak_rss=1388.2MiB
mr_par run 2 time=6.19s peak_rss=1383.5MiB
mr_par run 3 time=5.85s peak_rss=1575.0MiB
mr_par run 4 time=5.77s peak_rss=1494.2MiB
mr_par run 5 time=5.84s peak_rss=1324.1MiB
mr_par median time=5.85s peak_rss=1388.2MiB
$ .venv/bin/python scripts/profile.py mr_opt 5 -- .venv/bin/python scripts/mapreduce.py --workers 8 --job
p_opt --out p_opt
mr_opt run 1 time=5.28s peak_rss=718.4MiB
mr_opt run 2 time=5.70s peak_rss=713.9MiB
mr_opt run 3 time=5.70s peak_rss=669.8MiB
mr_opt run 4 time=6.06s peak_rss=725.5MiB
mr_opt run 5 time=5.54s peak_rss=735.3MiB
mr_opt median time=5.70s peak_rss=718.4MiB
$ .venv/bin/python scripts/profile.py duckdb 5 -- .venv/bin/python scripts/duckdb_skills.py
duckdb run 1 time=4.82s peak_rss=440.3MiB
duckdb run 2 time=4.70s peak_rss=417.0MiB
duckdb run 3 time=4.61s peak_rss=418.8MiB
duckdb run 4 time=4.56s peak_rss=477.1MiB
duckdb run 5 time=4.49s peak_rss=444.9MiB
duckdb median time=4.61s peak_rss=440.3MiB
$ .venv/bin/python scripts/profile.py s3_read 5 -- .venv/bin/python scripts/bench_read.py --workers 8
s3_read run 1 time=3.78s peak_rss=113.6MiB
s3_read run 2 time=3.72s peak_rss=111.9MiB
s3_read run 3 time=3.81s peak_rss=116.3MiB
s3_read run 4 time=3.67s peak_rss=113.4MiB
s3_read run 5 time=3.80s peak_rss=117.9MiB
s3_read median time=3.78s peak_rss=113.6MiB
```

Рис. 12. Выводы профилирования

Нижняя граница времени задается хранилищем. Одно чтение объекта без обработки занимает 3,78 с, это около 224 МБ/с. RustFS работает в Docker на одном SSD и собирает каждый блок из фрагментов EC 2+2. DuckDB тратит сверх этого чтения 0,83 с, оптимизированный MapReduce 1,92 с. У MapReduce сверху добавляются запуск процессов Python, фаза reduce и запись результатов.

DuckDB быстрее и экономнее по памяти, чем оптимизированный MapReduce. Он разбирает CSV векторно на нескольких потоках внутри одного процесса. Свой MapReduce запускает восемь процессов Python, и каждый занимает память под интерпретатор, boto3 и свои буферы. Зато модель MapReduce переживает отказ отдельной задачи, а задачи можно разнести по нескольким машинам. DuckDB работает в одном процессе на одной машине.

Бонусная оптимизация дала такие результаты. Базовая версия обрабатывает сплиты по-

следовательно, держит сплит и копии всех его строк в памяти, разбирает каждую строку CSV и выдает каждый ключ отдельно. Потоковое чтение кусками по 1 МиБ, предфильтр по байтам роли и комбайнер в одном процессе сократили время с 14,58 до 11,74 с, то есть на 19 процентов, а пик памяти с 278,0 до 163,2 МиБ, на 41 процент. Восемь процессов уменьшили время до 5,70 с, в 2,6 раза меньше базового. Память при этом выросла до 718,4 МиБ, потому что одновременно работают восемь процессов. По сравнению с простым распараллеливанием без других оптимизаций итоговая версия требует на 48 процентов меньше памяти, а время почти то же, 5,70 против 5,85 с. Эта разница меньше разброса между запусками. Комбайнер сократил данные шаффла со 191397 до 138585 записей, на 28 процентов.

6. Влияние навыков на зарплату

Для анализа берутся вакансии варианта с зарплатой в рублях, это 1255 из 1380 вакансий с зарплатой. Зарплата вакансии равна середине вилки или единственной указанной границе. Если зарплата указана до вычета НДФЛ, она умножается на 0,87, чтобы все суммы были на руки. Значения меньше 20000 рублей в месяц исключены как ошибки ввода, это 5 вакансий. Остается 1250 вакансий, у 995 из них есть навыки. Медиана по этим 995 вакансиям 160000 рублей. Опыт до трех лет требуют 56 процентов из них, дальше такие вакансии называются вакансиями для начинающих.

Сначала вакансии сравниваются по уровню зарплаты и набору навыков. Они делятся на четыре равные группы по зарплате.

Группа	Вакансий	Зарплата, рублей	Медиана	Навыков в среднем	Опыт до 3 лет, процентов	Частые навыки, вакансий
1	249	20000 и 100000	80000	6,2	92	git 114, android 106, ios 99, flutter 62, kotlin 62
2	249	100000 и 160000	130000	6,3	73	android 128, git 104, ios 96, flutter 75, kotlin 71
3	249	160000 и 250000	190000	6,1	47	android 118, git 103, ios 98, kotlin 86, flutter 61
4	248	250000 и 525000	291450	5,3	13	kotlin 120, android 112, git 63, ios 62, android sdk 60

Таблица 2. Вакансии по четвертям зарплаты, в столбце зарплаты наименьшее и наибольшее значение

В дорогих вакансиях меньше навыков, 5,3 против 6,2 в дешевых. Работодатель опытного разработчика называет несколько главных технологий, а в вакансии для начинающих перечисляет базовые инструменты. Git есть в 46 процентах вакансий нижней группы и в 25 процентах верхней. Kotlin выходит на первое место только в верхней группе, а Flutter пропадает из ее пяти частых навыков.

Прибавку от сочетания нужно считать от базы. Для пары база это более частый навык пары, для тройки самая частая из трех вложенных пар. Частоты берутся из результатов MapReduce в hh-lake. Прибавка равна медиане зарплаты вакансий с сочетанием минус медиана вакансий с базой. Медиана по нескольким вакансиям неустойчива, поэтому учитываются сочетания, которые есть минимум в 15 вакансиях с зарплатой. Таких 200 пар и 183 тройки.

```
SELECT s1 || ' + ' || s2 AS combo, b.n, base, m1.med AS base_med, b.med,
       b.med - m1.med AS uplift
FROM (SELECT m2.*, CASE WHEN fa.freq > fb.freq OR (fa.freq = fb.freq AND s1 < s2)
                        THEN s1 ELSE s2 END AS base
```

```
      FROM m2 JOIN f1 fa ON fa.skill = s1 JOIN f1 fb ON fb.skill = s2
      WHERE m2.n >= 15) b
JOIN m1 ON m1.skill = base;
```

Сочетание	С зарплатой	База и ее медиана	Медиана	Прибавка	Опыт до 3 лет
Пары, повышающие медиану					
android + kotlin coroutines	16	android, 158300	250000	91700, 58	19
coroutines + kotlin	20	kotlin, 187920	270000	82080, 44	20
jetpack compose + kotlin	21	kotlin, 187920	252300	64380, 34	43
dagger 2 + java	18	java, 156125	220000	63875, 41	33
android + hilt	16	android, 158300	221850	63550, 40	13
Тройки, повышающие медиану					
dagger 2 + java + kotlin	17	java + kotlin, 160000	260000	100000, 63	29
ios + solid + swift	17	ios + swift, 160000	250000	90000, 56	29
android + mvi + mvvm	15	android + mvvm, 150000	217500	67500, 45	13
android + kotlin + kotlin coroutines	16	android + kotlin, 187920	250000	62080, 33	19
android + dagger 2 + java	18	android + java, 160000	220000	60000, 38	33
Пары, понижающие медиану					
android studio + kotlin	21	kotlin, 187920	95700	−92220, −49	71
flutter + kotlin	18	kotlin, 187920	110000	−77920, −41	78
kotlin + mvp	24	kotlin, 187920	110000	−77920, −41	79
ios + json api	23	ios, 150000	80000	−70000, −47	70
android studio + java	17	java, 156125	90000	−66125, −42	76
Тройки, понижающие медиану					
android + android studio + kotlin	15	android + kotlin, 187920	90000	−97920, −52	67
android + kotlin + mvp	17	android + kotlin, 187920	110000	−77920, −41	88
android studio + java + kotlin	16	java + kotlin, 160000	87500	−72500, −45	75
android + android sdk + mvp	15	android + android sdk, 180000	110000	−70000, −39	87
android sdk + kotlin + mvp	17	android sdk + kotlin, 180000	110000	−70000, −39	88

Таблица 3. Итоговая таблица. Медианы и прибавка в рублях на руки, прибавка также в процентах от базы, опыт до 3 лет в процентах вакансий сочетания

Повышают зарплату навыки современного Android-стека. Это корутины Kotlin, Jetpack Compose, внедрение зависимостей Dagger 2 и Hilt, архитектура MVI. Пара android + kotlin coroutines поднимает медиану на 91700 рублей относительно вакансий с Android. В iOS к тому же ведет SOLID, тройка ios + solid + swift дает 250000 рублей против 160000 у ios + swift. В этих сочетаниях вакансий для начинающих от 13 до 43 процентов при 56 процентах в среднем по варианту.

Понижают зарплату Android Studio, MVP, JSON API и Flutter рядом с нативным языком. Причина не в самих навыках. Это среда разработки, устаревшая архитектура и базовые форматы, которые перечисляют в вакансиях для начинающих, их там от 67 до 88 процентов. Сочетание flutter + kotlin встречается в объявлениях младших кроссплатформенных разработчиков. Похожая картина у троек git + objective-c + swift и git + ios + objective-c, они понижают медиану на 61600 и 53950 рублей при 90 и 91 проценте вакансий для начинающих. Таблица показывает связь навыков с зарплатой в объявлениях, а не то, что изучение навыка само по себе меняет зарплату.

7. Региональный анализ

Выбраны Москва, Санкт-Петербург и Екатеринбург. После двух столиц в Екатеринбурге больше всего вакансий варианта с зарплатой, 66. В Новосибирске вакансий больше, 144 против 137, но с зарплатой только 46, а медиана по малой выборке неустойчива.

Показатель	Москва	Санкт-Петербург	Екатеринбург
Вакансий варианта	1752	592	137
Доля от всех вакансий варианта, процентов	46	16	4
Вакансий с зарплатой	522	179	66
Нижний квартиль зарплаты, рублей	140000	115000	102500
Медиана зарплаты, рублей	200000	160000	180000
Верхний квартиль зарплаты, рублей	277550	237450	300000
Опыт до 3 лет, процентов	32	42	58
Удаленная работа, процентов	42	34	32
Вакансий с навыками	1250	420	111
Навыков в вакансии в среднем	5,4	5,3	4,4

Таблица 4. Ключевые показатели регионов

	Москва	Санкт-Петербург	Екатеринбург
Навык 1	kotlin 39, 243600	android 43, 160000	android 33, 250000
Навык 2	android 39, 200000	kotlin 41, 186350	git 32, 125000
Навык 3	ios 30, 180000	ios 35, 160000	ios 30, 250000
Навык 4	git 27, 177110	swift 27, 190000	kotlin 29, 250000
Навык 5	swift 26, 200000	java 23, 160000	swift 27, 275000
Пара 1	android + kotlin 27, 224460	android + kotlin 31, 185000	android + kotlin 23, 250000
Пара 2	ios + swift 20, 190000	ios + swift 22, 195000	ios + swift 19, 300000
Пара 3	android sdk + kotlin 17, 226200	java + kotlin 21, 160000	java + kotlin 18, 240000
Тройка 1	android + android sdk + kotlin 13, 225330	android + java + kotlin 16, 160000	android + java + kotlin 14, 300000
Тройка 2	android + java + kotlin 12, 230000	android + android sdk + kotlin 12, 220000	android + android sdk + kotlin 13, 300000
Тройка 3	android sdk + java + kotlin 9, 227230	android sdk + java + kotlin 8, 150000	android sdk + java + kotlin 13, 240000

Таблица 5. Частые навыки, пары и тройки. В ячейке доля вакансий региона с навыками в процентах и медиана зарплаты в рублях

Таблицы построены одним SQL по тому же объекту S3. Ниже часть запроса для пар внутри региона, навыки и тройки считаются так же.

```
CREATE TABLE r AS SELECT seg.*, sal.salary FROM seg LEFT JOIN sal USING (vacancy_id)
WHERE seg.area IN ('Москва', 'Санкт-Петербург', 'Екатеринбург');
CREATE TABLE e AS SELECT vacancy_id, area, salary, unnest(skills) AS skill FROM r;
SELECT a.area, a.skill || ' + ' || b.skill AS combo, count(*) AS vacancies,
       count(a.salary) AS n_salary, median(a.salary) AS median_salary
FROM e a JOIN e b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
GROUP BY ALL;
```

Москва дает почти половину вакансий варианта и самую высокую медиану, 200000 рублей. Вакансий для начинающих там меньше всего, 32 процента, а удаленной работы больше всего, 42 процента. Kotlin в Москве первый навык, и вакансии с ним дают медиану 243600 рублей. Столичный рынок ищет опытных Android-разработчиков на современном стеке.

В Санкт-Петербурге медиана на 20 процентов ниже московской, 160000 рублей. Вакансий для начинающих больше, 42 процента, а связка Android и Kotlin встречается чаще, чем в Москве, в 31 проценте вакансий. Петербургский рынок сильнее смещен к Android и к Java рядом с Kotlin.

Екатеринбург самый неоднородный. Вакансий для начинающих 58 процентов, навыков в вакансии меньше всего, 4,4, а разброс зарплат самый широкий, от 102500 до 300000 рублей между квартилями. Медиана 180000 рублей выше петербургской за счет вакансий с опытом от 3 до 6 лет, их медиана 300000 рублей. С опытом от 1 до 3 лет там предлагают 155000 рублей, в столицах 150000. В Екатеринбурге чаще, чем в столицах, встречаются Flutter, 19 процентов вакансий, Dart, 16 процентов, ООП, 17 процентов, и git, 32 процента. Это признаки кроссплатформенных проектов и вакансий для начинающих. Медианы по навыкам Екатеринбурга основаны на 3 и 18 вакансиях с зарплатой и показывают только направление.

Исходный код всех программ, логи, результаты и этот отчет опубликованы в репозитории `https://github.com/Gegerout/data-engineering-lab3`. Команды запуска описаны в README репозитория.